

A Comparative Study on the Effectiveness of Generative AI Tools and Pair Programming in College Students' Programming Learning

Hui He*
Jiangnan University
Wuhan, China
hehui@jhun.edu.cn

*Corresponding author

Xing Li
Jiangnan University
Wuhan, China
lixing@jhun.edu.cn

Abstract—With the rapid development of artificial intelligence technology in the field of education, AI has shown great potential in programming education. However, the traditional pair programming method cannot provide immediate feedback and has certain limitations. This study proposes an innovative AI-assisted learning method, aiming to enhance students' programming learning experience and efficiency through the assistance of AI. This study employed a hierarchical guided AI tool based on large language models and adopted a quasi-experimental design to investigate the impact of AI methods on students' academic performance, self-efficacy, and learning interest. A study was conducted on 80 undergraduate students of the C language programming course. Two classes were randomly assigned to AI assisted pair programming (n=38) and human-human pair programming (n=42). The research results show that artificial intelligence assistance can effectively improve students' programming performance, self-efficacy and programming interest. This research provides strong practical guidance for integrating AI-assisted learning into programming courses in colleges and universities.

Keywords—AI-assisted programming, programming performance, self-efficacy, programming interest

I. INTRODUCTION

In today's intelligent era, programming ability has become a core library resource for talents in the 21st century. It can cultivate students' logical thinking, innovation ability and problem-solving ability. Therefore, institution of higher learning incorporate programming courses into the curricula of various majors. However, in traditional programming education, learning programming is a challenging and complex process. During the process of writing programs, students often encounter problems such as programming errors and difficulties in understanding algorithms, which have become obstacles for them to learn programming. Among them, Compilation and running errors are the mistakes that students inevitably encounter during the programming learning process. It is difficult for learners to debug the errors in the written programs. And they often need certain support and guidance. Therefore, how to solve the predicament in programming education through the innovative reconstruction of teaching paradigms is an urgent problem to be solved at present.

In recent years, generative artificial intelligence technology has been widely applied in various fields. Large language models can assist learning and teaching through the input,

interaction and generation of natural language texts, providing more convenient and efficient learning tools for students and teachers. In addition, generative artificial intelligence tools have the ability to provide immediate feedback and personalized guidance. They can quickly offer solutions to the problems that learners encounter in programming and provide them with support and guidance. This helps students quickly identify and correct programming errors, and gain a deeper understanding and mastery of programming knowledge. However, the application effect of generative artificial intelligence in programming education has not been fully studied yet.

II. LITERATURE REVIEW

A. Self-efficacy and Interest in Programming Education

Self-efficacy and learning interest undertake important mediating factors in the learning process, and they can directly or indirectly affect learning performance^[1]. Self-efficacy is the degree of an individual's confidence in their ability to perform a given behavior or complete a specific task^[2]. Learners with different levels of self-efficacy will show different attitudes when facing difficulties. People with low self-efficacy often regard challenges as threats. They tend to become discouraged and choose to give up. Those with high self-efficacy tend to regard challenges as incentives. They continuously invest energy and actively seek solutions to overcome obstacles^[3].

Programming self-efficacy refers to learners' confidence in using the necessary skills to complete programming tasks^[4]. Previous studies have shown that individuals with a high sense of self-efficacy tend to set higher expectations for their own abilities and devote more efforts to meeting challenges. Therefore, they are more likely to successfully solve more complex problems^[5].

Interest is an important component of learning motivation. In the implementation process of programming courses, the novel teaching design is usually emphasized to stimulate students' interest in learning. In terms of programming learning, based on the self-determination theory, it is believed that students who are interested in programming may acquire stronger programming abilities^[4].

B. Pair Programming and AI-supported Programming

Pair programming is a form of cooperative learning, which refers to two people sharing one computer and taking turns to play the roles of "pilot" and "navigator"^[6]. The driver is mainly

responsible for writing the code and implementing the design ideas, while the navigator constantly reviews the code of his companions, supervises the overall logic and provides constructive suggestions. When encountering difficult problems, they can discuss them together. If necessary, they can also follow the agreement to swap the responsibilities of the driver and the navigator. At first, pair programming was mainly applied in enterprise software development. Subsequently, it was gradually used in the programming courses of colleges and universities and became the focus of attention in both the academic and industrial fields. Existing studies have shown that pair programming can effectively improve programming ability [7], and it is conducive to enhancing the motivation for programming learning [8] and computational thinking [9]. However, pair programming also has certain limitations, such as the inability to provide immediate feedback and personalized assistance.

The emergence of large language models makes it possible to solve the above limitations. Artificial intelligence, as a virtual partner, can provide immediate feedback. Students can interact with the virtual partner at any time, and AI can provide algorithmic solutions for students [10]. Kimmel pointed out that generative AI has a significant effect in enhancing students' programming ability [11]. Fan et al. found that AI-assisted programming can stimulate learning motivation and alleviate programming anxiety [12]. However, in existing studies, students directly request complete answers from large language models, which easily leads to excessive reliance on AI and weakens learners' willingness to explore independently and think deeply. Therefore, we should introduce a more refined "hierarchical" guiding strategy. Through knowledge Q&A, code error correction and debugging, as well as code interpretation and other links, students are helped to build programming cognition step by step, rather than relying solely on large language models to output the final answer.

III. EXPERIMENTAL DESIGN

A. Participant

This study aims to explore the impact of AI assistance on the programming ability of college students. Our team conducted a quasi-experimental study at a university in Wuhan to evaluate the effectiveness of applying AI tools to programming learning. In this study, students from two classes majoring in Digital Media Technology were selected as participants and formed the experimental group and the control group respectively. All students are taking the C language programming course. To test the homogeneity of the groups, a pre-test of c language programming ability was conducted for the two groups of students before the experiment began. Analysis of variance showed that the baseline levels of the two groups were statistically consistent, indicating homogeneity between the two groups ($F=0.075$, $p=0.785 > 0.05$).

Ultimately, this experiment was divided into two groups: the AI group ($n=38$), where AI tools were used to assist in programming learning and the code was debugged through AI feedback. The PP group ($n=42$) used the pair programming method for programming learning. This experiment was conducted in a laboratory of an university, with C language programming as the main learning resource, aiming to explore

the influence of AI tools on the programming learning process and learning outcomes.

B. Experimental Procedure

Teachers adopt a blended teaching mode in theoretical teaching. Before class, teachers release learning tasks and resources through online learning platform, and students complete previewing and task sheets independently. During the class, the teacher focuses on analyzing the task results submitted by the students, discusses the core concepts and programming design ideas, and guides the students to construct a knowledge system at the theoretical level.

In the experimental and practical sections, students carry out program design following the process of "task analysis → process design → code writing → debugging and optimization": First, they conduct requirement analysis and process definition, and they write the initial code based on the design concept. Then students run the test, use breakpoints, logs and other means to locate syntax or logical errors, and continuously refactor and optimize performance until the program functionality meets the standards.

For the AI group, a refined hierarchical guidance strategy was innovatively introduced. As shown in Figure 1, when students encounter difficulties, the AI virtual partner provides support in three major modules. The first function is Knowledge Q&A. It can answer basic concepts and algorithm principles at any time. The second function is Code error correction and debugging. It can automatically detect and mark syntax and logic vulnerabilities, and provide repair suggestions. The last function is code interpretation and providing examples. It can explain the program execution flow in natural language and generate similar function examples to help students gradually understand and master core skills. For the PP group, students in the paired programming group collaborate and communicate together based on tasks to complete code design and implementation, code debugging and optimization.

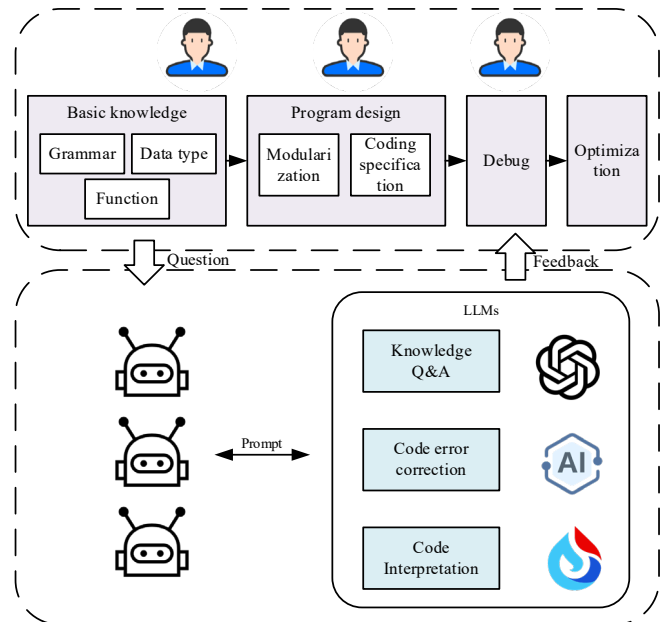


Fig. 1. AI supported programming method

The experimental process is shown in Figure 2. The entire experiment lasted for 8 weeks, with 90 minutes of experimental time each week. In the first week, we tested the students' academic performance, self-efficacy and learning attitude to provide baseline data for the subsequent analysis. Students in the experimental group became familiar with the AI tool and used the AI tool to assist in programming, while students in the control group became familiar with the pair programming environment and completed programming through peer communication. The group training ensures that both groups of students are fully prepared in their respective learning environments. From the second to the seventh week, students learned the basic knowledge of C language structure and programming. The experimental group used AI to ask questions, write code and debug code, while the control group communicated with their companion to complete code writing and debugging. In the eighth week, we conducted a post-test on students' self-efficacy and programming interest. Meanwhile, we conduct post-tests on students' learning performance to comprehensively assess the impact of using AI tools on students' programming learning.

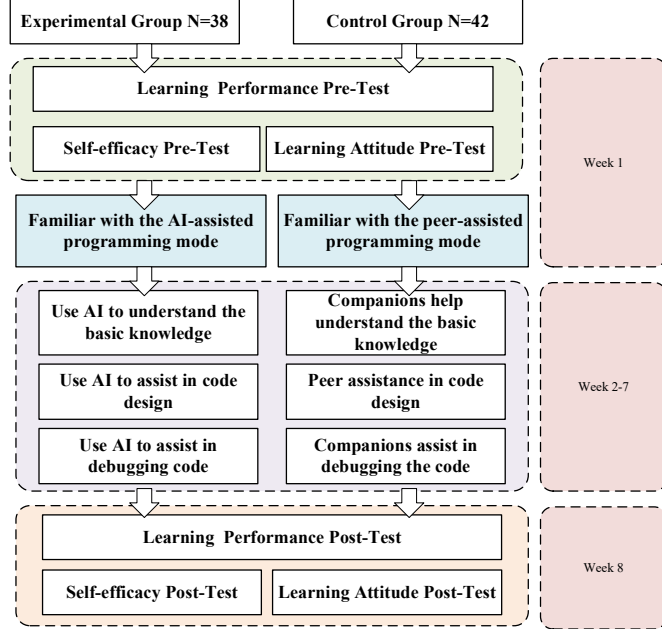


Fig. 2. Experimental Procedure

C. Measurement Tools

The tools used in this study include tests and questionnaires for programming proficiency, self-efficacy and programming interest. The pre-test and post-test of programming proficiency were jointly developed by two experienced teachers who teach the C language programming course, referring to the basic concept framework of the course. This test is mainly used to assess students' initial proficiency and final programming scores. It consists of 9 multiple-choice questions and 1 programming question, with a total score of 50 points. The overall Cronbach's alpha is 0.82.

The self-efficacy questionnaire was modified based on the measurement method developed by Kong^[4] and included four items. For example, I am very confident in my C language

programming ability. The questionnaire adopts the 5-point Likert from 1 = Strongly disagree to 5 = Strongly agree. The overall Cronbach's alpha for this questionnaire is 0.94.

The learning interest Questionnaire is modified based on the measurement method developed by Mazer (2013)^[13] and included four items. The questionnaire adopts the 5-point Likert from 1 = Strongly disagree to 5 = Strongly agree. For example, C language Programming makes me very excited. The overall Cronbach's alpha for this questionnaire is 0.93.

IV. RESULT

A. Descriptive Statistics

The sample of this study was 80 undergraduate students (aged 19.34 years, SD= 1.23) taking the C language programming course. Baseline measurements indicated that the initial programming ability, programming interest, and self-efficacy levels of all groups were comparable.

B. Learning Performance

The descriptive statistical results of learning performance are shown in Table 1. In order to analyze the differences in the scores of the pre-test and post-test among different groups, we conducted paired sample t-tests on the pre-test scores and post-test scores of programming performance in the experimental group and the control group respectively. The results showed that the post-test score of the experimental group was significantly greater than the pre-test score ($t=-6.478$, $p<0.001$). The score of the post-test and the score of the pre-test in the control group were close ($t=0.087$, $p>0.05$). This result indicates that AI-assisted programming has a significant positive effect on the overall improvement of college students' programming ability.

In order to analyze the differences in the scores of post-tests among different groups, we conducted an independent sample t-test on the post-test scores of learning performance in different groups. The results showed that there were significant differences in the post-test scores among different groups ($t=5.496$, $p<0.001$). The results indicate that AI-assisted programming has a positive effect on the overall improvement of college students' programming ability.

TABLE I. MEANS AND STANDARD DEVIATIONS OF LEARNING PERFORMANCE

Group	Learning Performance	M	SD	Difference
AI Group	Pre-Test	24.34	5.83	10
	Post-Test	34.34	8.31	
Peer Group	Pre-Test	25.59	5.86	0.16
	Post-Test	25.47	6.02	

C. Programming Interest

To examine the influence of different learning patterns on students' self-efficacy, this study adopted one-way covariance analysis (ANCOVA), with post-test result as the dependent variable, group as the independent variable, and pre-test result as the covariate. Before the analysis, a premise test was conducted. The result of Levene's homogeneity of variance test was not significant ($F= 3.511$, $p >0.05$), indicating that the variances between groups were approximately equal and satisfying the homogeneity of variance hypothesis. The result of

the homogeneity of regression slopes was also not significant ($F(1,76) = 1.523, p > 0.05$), indicating that the influence trend of the covariates was consistent in different groups, which met the prerequisite conditions for ANCOVA analysis.

The results of covariance analysis showed that the group had a significant influence on the post-test scores of students ($F(1,77) = 18.05, p < .001$), while the pre-test score of the covariate had no significant influence on the post-test score ($F(1,77) = 0.002, p = .967$). This indicates that, while controlling for the influence of the pre-test scores, there are significant differences in the post-test scores among students in different groups. In addition, as shown in Table 2, the average post-test score of the experimental group was 3.54, which was significantly higher than 3.22 of the control group. The difference between the two groups was statistically significant ($p < .001$), with a large effect size ($\eta^2 = 0.171$). This result indicates that the adopted experimental learning model is more effective in enhancing students' self-efficacy.

TABLE II. MEANS AND STANDARD DEVIATIONS OF PROGRAMMING INTEREST

Group	Interest test	M	SD
AI Group	Pre-Test	3.20	0.32
	Post-Test	3.54	0.36
Peer Group	Pre-Test	3.20	0.31
	Post-Test	3.22	0.30

D. Self-efficacy

To examine the influence of different learning patterns on students' self-efficacy, this study adopted one-way covariance analysis (ANCOVA), with post-test result as the dependent variable, group as the independent variable, and pre-test result as the covariate. Before the analysis, a premise test was conducted first. The result of Levene's homogeneity of variance test was not significant ($F = 1.377, p = .244 > 0.05$), indicating that the variances between groups were approximately equal and satisfying the homogeneity of variance hypothesis. The result of the homogeneity of regression slopes was not significant ($F = 0.005, p = .942 > 0.05$), indicating that the influence trend of the covariates was consistent in different groups, which met the prerequisite conditions of ANCOVA and could be further analyzed.

The results of covariance analysis showed that the group had a significant impact on the post-test score of students ($F(1,77) = 101.17, p < .001$), while the main effect of the covariate pre-test score was not significant ($F(1,77) = 3.396, p > 0.05$), indicating that under the condition of controlling the influence of the pretest score, There are still significant differences in the post-test scores among students in different groups. In addition, as shown in Table 3, the average post-test score of the experimental group was 3.63, which was significantly higher than 3.19 of the control group. The difference between the two groups was statistically significant ($p < .001$), with a large effect size ($\eta^2 = 0.575$). This result indicates that the adopted experimental learning model is more effective in enhancing students' self-efficacy.

TABLE III. MEANS AND STANDARD DEVIATIONS OF SELF-EFFICACY

Group	SE Test	M	SD
AI Group	Pre-Test	3.17	0.21

PP Group	Post-Test	3.63	0.18
	Pre-Test	3.16	0.23
	Post-Test	3.19	0.22

V. DISCUSSION, LIMITATIONS AND CONCLUSION

This study aims to explore the impact of artificial intelligence tools on the performance of college students in programming courses. The experimental group conducted programming learning with the aid of AI tools, while the control group learned through paired programming. Our research results indicate that the use of AI tools to assist learning has a significant positive impact on students' programming ability, programming interest and self-efficacy.

First of all, learners who use AI tools to assist in programming perform better in academic performance than those who do pair programming. This is because AI tools can provide personalized learning support, making the programming process more efficient. In pair programming learning, learners can obtain feedback from their peers. However, the way of feedback lacks personalization for complex problems.

Secondly, the self-efficacy of the AI group assisted in programming with AI tools was higher than that of the paired programming group. This is because AI tools can quickly interpret code and correct errors. Such timely assistance effectively reduces learners' frustration^[14], thereby enhancing their sense of self-efficacy. AI tools help students respond positively when facing complex tasks and enhance their learning motivation. In pair programming learning, when facing complex tasks, one often falls into the "waiting mode", which inhibits the growth of self-efficacy.

Finally, the programming interest of the AI group assisted by AI tools was higher than that of the paired programming group. This is because AI tools can continuously interact with learners, and the immediate feedback enhances students' programming skills. Their successful experiences increase their interest in learning. However, the feedback is delayed and the feedback efficiency is low in pair programming learning.

In general, the AI-assisted programming method proposed in this study can help students improve their programming skills, enhance their sense of self-efficacy and programming interest.

While this study highlights the positive impact of AI tools on students' programming performance, self-efficacy, and interest, several limitations should be considered. Firstly, the observed improvements may be influenced by the novelty effect, where the initial excitement of using a new technology temporarily boosts engagement and performance. Over time, as the novelty diminishes, these effects may wane, potentially affecting long-term outcomes. Secondly, the study's scope is limited to a C language programming course at a single university, involving students with specific backgrounds. This narrow context may limit the generalizability of the findings to other programming languages, educational settings, or diverse student populations.

To enhance the applicability of future research, studies should be conducted across various educational environments, programming languages, and student demographics. Additionally, longitudinal studies are recommended to assess

the sustained impact of AI tools on learning outcomes beyond the initial period of novelty.

ACKNOWLEDGMENT

The phased or concluding achievements of the Wuhan Education Science Planning Project(2024C028). This work was supported by the university-level teaching reform research project "Artificial Intelligence + Education" of Jiangnan University.

REFERENCES

- [1] Lee, W., Lee, M. J., & Bong, M. (2014). Testing interest and self-efficacy as predictors of academic self-regulation and achievement. *Contemporary educational psychology*, 39(2), 86-99.
- [2] Bandura, A. (1982). Self-efficacy mechanism in human agency. *American psychologist*, 37(2), 122.
- [3] Bandura, A. (1997). Self-efficacy: The exercise of control.
- [4] Kong, S. C., Chiu, M. M., & Lai, M. (2018). A study of primary school students' interest, collaboration attitude, and programming empowerment in computational thinking education. *Computers & education*, 127, 178-189.
- [5] Bandura, A. (2023). Cultivate self - efficacy for personal and organizational effectiveness. *Principles of Organizational Behavior: The Handbook of Evidence - Based Management 3rd Edition*, 113-135.
- [6] Williams, L., Wiebe, E., Yang, K., Ferzli, M., & Miller, C. (2002). In support of pair programming in the introductory computer science course. *Computer Science Education*, 12(3), 197-212.
- [7] Dongo, T. A., Reed, A. H., & T O'Hara, M. (2016). Exploring pair programming benefits for MIS majors. *Journal of Information Technology Education. Innovations in Practice*, 15, 223.
- [8] McDowell, C., Werner, L., Bullock, H. E., & Fernald, J. (2006). Pair programming improves student retention, confidence, and program quality. *Communications of the ACM*, 49(8), 90-95.
- [9] Wei, X., Lin, L., Meng, N., Tan, W., & Kong, S. C. (2021). The effectiveness of partial pair programming on elementary school students' computational thinking skills and self-efficacy. *Computers & education*, 160, 104023.
- [10] Bird, C., Ford, D., Zimmermann, T., Forsgren, N., Kalliamvakou, E., Lowdermilk, T., & Gazit, I. (2022). Taking Flight with Copilot: Early insights and opportunities of AI-powered pair-programming tools. *Queue*, 20(6), 35-57.
- [11] Kimmel, B., Geisert, A. L., Yaro, L., Gipson, B., Hotchkiss, R. T., Osae-Asante, S. K., ... & Yamaguchi, C. (2024, May). Enhancing programming error messages in real time with generative AI. In *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems* (pp. 1-7).
- [12] Fan, G., Liu, D., Zhang, R., & Pan, L. (2025). The impact of AI-assisted pair programming on student motivation, programming anxiety, collaborative learning, and programming performance: a comparative study with traditional pair programming and individual approaches. *International Journal of STEM Education*, 12(1), 16.
- [13] Mazer, J. P. (2013). Validity of the student interest and engagement scales: Associations with student learning outcomes. *Communication Studies*, 64(2), 125-140.
- [14] Johnson, D. M., Doss, W., & Estepp, C. M. (2024). Using ChatGPT with novice Arduino programmers: Effects on performance, interest, self-efficacy, and programming ability. *Journal of Research in Technical Careers*, 8(1), 1.